

PWA Web Push 알림 기능 구현 가이드

본 문서는 웹 브라우저 및 모바일 기기(아이폰 iOS 16.4+ 포함)에서 백그라운드 푸시 알림을 수신할 수 있도록 ****PWA Web Push 기능****을 구현하는 절차와 핵심 코드를 정리한 가이드입니다.

1. 동작 아키텍처 및 원리

Web Push는 국제 표준 웹 사양을 따르며, 브라우저 제조사(Google, Apple, Mozilla 등)가 운영하는 푸시 서버(Push Service)를 거쳐 작동합니다.

...

[동작 흐름]

1. 사용자 기기(브라우저)에서 알림 권한 허용 → 브라우저 내 '서비스 워커(sw.js)' 등록
2. VAPID Public Key를 사용하여 브라우저에서 '구독 정보(Subscription)' 발급
3. 발급된 구독 정보를 백엔드 서버로 전송 및 DB 저장
4. 알림 상황 발생 시 백엔드에서 VAPID Private Key로 서명하여 제조사 푸시 서버로 알림 발송 요청
5. 제조사 푸시 서버가 사용자 기기에 알림 패킷을 보냄 → 서비스 워커가 감지하여 알림 노출

...

2. 🔑 1단계: VAPID 키 쌍 생성

보안 통신을 위해 비대칭 키 쌍(Public Key, Private Key)이 필요합니다.

키 발급 방법 (Node.js 환경)

터미널에서 아래 명령을 실행하여 키 쌍을 생성합니다.

```
```bash
npx web-push generate-vapid-keys
```
```

* ****Public Key****: 프론트엔드가 구독을 신청할 때 사용합니다. (외부 공개 가능)

* ****Private Key****: 백엔드 서버에서 푸시 메시지에 서명할 때 사용합니다. (보안 노출 주의)

3. 🧩 2단계: 프론트엔드 구현

1) 서비스 워커 파일 작성 (`public/sw.js`)

브라우저 루트 폴더(예: public 디렉토리)에 서비스 워커 파일을 배치합니다.

이 파일은 백그라운드에서 푸시 이벤트를 수신하여 알림을 노출합니다.

```
````javascript
// public/sw.js

// 1. 서비스 워커 설치 및 활성화 즉시 제어권 획득
self.addEventListener("install", (event) => {
 self.skipWaiting();
});

self.addEventListener("activate", (event) => {
 event.waitUntil(self.clients.claim());
});

// 2. 푸시 메시지 수신 이벤트 처리
self.addEventListener("push", (event) => {
 let data = {
 title: "알림 타이틀",
 body: "알림 내용을 확인하세요.",
 url: "/",
 };

 if (event.data) {
 try {
 data = { ...data, ...event.data.json() };
 } catch (error) {
 data.body = event.data.text();
 }
 }

 event.waitUntil(
 self.registration.showNotification(data.title, {
 body: data.body,
 icon: "/icons/icon-192.png", // 알림 아이콘 경로
 badge: "/icons/icon-192.png", // 상태바 아이콘 경로
 data: {
 url: data.url, // 클릭 시 이동할 URL 저장
 },
 })
);
});

// 3. 알림 클릭 시 특정 URL로 창 열기 또는 포커스 이동
self.addEventListener("notificationclick", (event) => {
 event.notification.close(); // 알림 닫기
 const targetUrl = event.notification.data?.url || "/";
```

```

 event.waitUntil(
 self.clients.matchAll({ type: "window", includeUncontrolled:
true }).then((clientList) => {
 // 이미 열려 있는 해당 사이트 창이 있다면 포커스
 for (const client of clientList) {
 if ("focus" in client) {
 client.navigate(targetUrl);
 return client.focus();
 }
 }
 // 열려 있는 창이 없다면 새로 열기
 if (self.clients.openWindow) {
 return self.clients.openWindow(targetUrl);
 }
 })
);
 });
 ...

```

### 2) 서비스 워커 등록 및 구독 생성 로직 (React/Next.js 예시)

웹 프론트엔드 코드 내에서 브라우저 권한을 얻고 푸시를 연동하는 로직입니다.

```

````typescript
// 유틸리티 함수: Base64 VAPID Key를 Uint8Array로 변환
function urlBase64ToUint8Array(base64String: string) {
  const padding = "=".repeat((4 - (base64String.length % 4)) % 4);
  const base64 = (base64String + padding).replace(/-/g, "+").replace(/_/g,
"/");
  const rawData = window.atob(base64);
  const outputArray = new Uint8Array(rawData.length);
  for (let i = 0; i < rawData.length; ++i) {
    outputArray[i] = rawData.charCodeAt(i);
  }
  return outputArray;
}

// 푸시 알림 권한 허용 및 서버 등록 함수
async function enablePushNotification() {
  if (!("Notification" in window) || !("serviceWorker" in navigator)
|| !("PushManager" in window)) {
    alert("이 브라우저/기기에서는 알림 기능을 지원하지 않습니다.");
    return;
  }

  // 1. 알림 권한 요청
  const permission = await Notification.requestPermission();
  if (permission !== "granted") {

```

```

    alert("알림 권한이 허용되지 않았습니다.");
    return;
}

try {
    // 2. 서비스 워커 등록
    await navigator.serviceWorker.register("/sw.js");
    const registration = await navigator.serviceWorker.ready;

    // 3. 이미 존재하는 구독 정보 확인
    let subscription = await registration.pushManager.getSubscription();

    // 4. 새로운 구독 등록 (VAPID Public Key 필요)
    if (!subscription) {
        const vapidPublicKey = "YOUR_VAPID_PUBLIC_KEY"; // 여기에 발급받은
        Public Key 삽입
        subscription = await registration.pushManager.subscribe({
            userVisibleOnly: true,
            applicationServerKey: urlBase64ToUint8Array(vapidPublicKey),
        });
    }

    // 5. 서버 API 호출하여 subscription 정보 저장
    await fetch("/api/push-subscriptions", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            subscription: subscription, // 이 객체가 push endpoint와 인증키를
            포함함
            userAgent: navigator.userAgent,
        }),
    });

    alert("알림 설정이 성공적으로 완료되었습니다!");
} catch (error) {
    console.error("푸시 설정 오류:", error);
}
}

```

4. 3단계: 백엔드 구현

백엔드는 프론트엔드로부터 받은 **구독 정보(`subscription`)**를 데이터베이스에 보관해 두었다가, 필요할 때 이 정보로 푸시 서버에 요청을 전송합니다.

1) DB 저장 객체 구조

클라이언트가 보내주는 `subscription`은 아래와 같은 JSON 형태이며, 데이터베이스 테이블에 그대로 텍스트나 JSON으로 저장해야 합니다.

```
```json
{
 "endpoint":
 "https://updates.push.services.mozilla.com/wpush/v2/gAAAAAB...",
 "expirationTime": null,
 "keys": {
 "auth": "Authentication Secret Key (Base64)",
 "p256dh": "Elliptic Curve Public Key (Base64)"
 }
}
```
```

2) 푸시 발송 코드 구현 예시

Python 백엔드 예시 (`pywebpush` 라이브러리 사용)

```
```bash
pip install pywebpush
```

```python
from pywebpush import webpush, WebPushException
import json

def send_web_push(subscription_info: dict, message_data: dict,
private_key: str, admin_email: str):
 """
 subscription_info: DB에 저장된 클라이언트 subscription JSON 객체
 message_data: 전송할 데이터 {"title": "제목", "body": "내용", "url":
"/"}
 private_key: VAPID Private Key
 admin_email: mailto:admin@example.com (푸시 서비스 연락처 필수)
 """
 try:
 webpush(
 subscription_info=subscription_info,
 data=json.dumps(message_data),
 vapid_private_key=private_key,
 vapid_claims={
 "sub": f"mailto:{admin_email}"
 }
)
 return True
 except WebPushException as ex:
 # 만약 410 Gone 에러가 리턴되면, 만료되거나 차단된 구독이므로 DB에
 # 삭제 처리

```

```

 print("푸시 발송 에러:", ex)
 return False
 ...

```

#### ● Node.js 백엔드 예시 (`web-push` 라이브러리 사용)

```

```bash
npm install web-push
...

```javascript
const webpush = require('web-push');

// 초기 설정
webpush.setVapidDetails(
 'mailto:admin@example.com',
 'YOUR_VAPID_PUBLIC_KEY',
 'YOUR_VAPID_PRIVATE_KEY'
);

function sendPushNotification(subscription, payload) {
 // payload: JSON.stringify({ title: '제목', body: '내용', url: '/' })
 webpush.sendNotification(subscription, payload)
 .then(response => console.log('알림 전송 성공', response))
 .catch(error => {
 if (error.statusCode === 410) {
 // 만료된 구독 정보이므로 DB 제거 처리
 console.log('만료된 구독 정보:', subscription.endpoint);
 } else {
 console.error('발송 실패:', error);
 }
 });
}
...

```

---

## ## 5. ⚠️ 모바일 및 iOS(아이폰) 적용을 위한 필수 체크리스트

1. **\*\*HTTPS 보안 프로토콜 필수\*\***:
  - \* 로컬 환경을 제외하고는 반드시 SSL이 적용된 HTTPS 프로토콜에서만 구동됩니다.
2. **\*\*PWA 홈 화면 추가 필수 (아이폰만 해당)\*\***:
  - \* 아이폰 사용자는 반드시 Safari 브라우저에서 **\*\*[공유] -> [홈 화면에 추가]\*\***를 눌러 설치 후 앱 형태로 실행해야 권한 요청과 수신이 가능합니다.
3. **\*\*Web Manifest 파일 추가 (`manifest.json`)\*\***:
  - \* 앱 설치 및 런처 형태 정의를 위해 프로젝트 루트(or public) 폴더에 `manifest.json` 설정이 필수적으로 탑재되어 있어야 합니다.